

Web Technologies

Complete Study Notes

Unit 2: Dynamic Documents with JavaScript

Unit 3 (Part A): Introduction to PHP

Unit 3 (Part B): XML

Descriptive Exam Preparation

UNIT 2

Dynamic Documents with JavaScript

1. Introduction to Dynamic Documents

A dynamic XHTML document is one where content, style, or structure can change after the page has loaded without a page reload, using JavaScript and the DOM.

Using the DOM, JavaScript can:

- Move elements on the page
- Change styles (colors, fonts)
- Toggle element visibility
- Modify content dynamically

2. Element Positioning (CSS-P)

CSS Positioning provides tools to place elements precisely on a web page. The position property specifies the position mode.

Value	Description
static	Default. Elements placed left-to-right, top-to-bottom. top/left ignored.
relative	Moves element relative to its normal static position.
absolute	Places element at exact coordinates relative to containing element.

top and left Properties

- Positive top -> pushes element DOWN
- Negative top -> pushes element UP
- Positive left -> pushes element RIGHT
- Negative left -> pushes element LEFT

Example: Absolute Positioning

```
<p style="position:absolute; left:100px; top:200px">
  This text appears at exactly (100, 200)
</p>
```

>> Exam Tip: To dynamically move elements with JavaScript, position must be absolute or relative, never static.

3. Moving Elements with JavaScript

JavaScript repositions elements by changing their top and left CSS properties at runtime.

Example Program: mover.html

```
<!DOCTYPE html>
<html>
<head><script>
function moveImage() {
  var x = document.getElementById('xCoord').value;
  var y = document.getElementById('yCoord').value;
  var img = document.getElementById('myImage');
  img.style.left = x + 'px';
```

```

    img.style.top = y + 'px';
  }
</script></head>
<body>
  <img id='myImage' src='plane.jpg'
    style='position:absolute; left:50px; top:50px;' />
  X: <input type='text' id='xCoord' value='50' />
  Y: <input type='text' id='yCoord' value='50' />
  <button onclick='moveImage()'>Move</button>
</body></html>

```

- Access element's style node via `element.style`
- Assign values like '100px' -- must include unit px

4. Element Visibility

Value	Effect
visible	Element is shown
hidden	Element is invisible but still occupies space in the layout

Example Program: showHide.html

```

<script>
function toggleVisibility() {
  var img = document.getElementById('myImage');
  if (img.style.visibility === 'hidden') {
    img.style.visibility = 'visible';
  } else {
    img.style.visibility = 'hidden';
  }
}
</script>
<img id='myImage' src='logo.png'
  style='position:absolute; visibility:visible;' />
<button onclick='toggleVisibility()'>Show/Hide</button>

```

>> Exam Tip: `display:none` removes the element from layout (no space). `visibility:hidden` hides but keeps space.

5. Changing Colors and Fonts

CSS Property (JS)	Example Value
<code>style.backgroundColor</code>	'yellow'
<code>style.color</code>	'red'
<code>style.fontFamily</code>	'Arial'
<code>style.fontSize</code>	'18px'
<code>style.fontWeight</code>	'bold'
<code>style.fontStyle</code>	'italic'

Example Program: dynColors.html

```

<script>
function changeColors() {

```

```
var bg = document.getElementById('bgColor').value;
var fg = document.getElementById('fgColor').value;
document.body.style.backgroundColor = bg;
document.getElementById('myText').style.color = fg;
}
</script>
<p id='myText'>Watch my color change!</p>
Background: <input type='text' id='bgColor' onchange='changeColors()' />
Text Color: <input type='text' id='fgColor' onchange='changeColors()' />
```

Changing Fonts on Hover (dynLink.html)

```
<a id='lnk' href='#'
  onmouseover="lnk.style.fontSize='20px'; lnk.style.fontWeight='bold';"
  onmouseout="lnk.style.fontSize='14px'; lnk.style.fontWeight='normal';">
  Hover Over Me
</a>
```

6. Dynamic Content

Dynamic content means changing what is displayed without reloading. The value property of text areas or innerHTML of elements can be updated using mouse events.

Example Program: dynValue.html

```
<script>
var helpMessages = [
  'Enter your first name here.',
  'Enter your last name here.',
  'Enter your email address.'
];
function showHelp(index) {
  document.getElementById('helpBox').value = helpMessages[index];
}
function clearHelp() {
  document.getElementById('helpBox').value = '';
}
</script>
First Name: <input type='text' onmouseover='showHelp(0)' onmouseout='clearHelp()' />
Last Name: <input type='text' onmouseover='showHelp(1)' onmouseout='clearHelp()' />
Email: <input type='text' onmouseover='showHelp(2)' onmouseout='clearHelp()' />
Help: <textarea id='helpBox' rows='2' cols='40'></textarea>
```

7. Stacking Elements (z-index)

The z-index CSS property controls layering of overlapping elements. Higher z-index = appears on top.

Example Program: stacking.html

```
<script>
function bringToFront(id) {
  document.getElementById('box1').style.zIndex = '1';
  document.getElementById('box2').style.zIndex = '1';
  document.getElementById(id).style.zIndex = '10';
}
</script>
<div id='box1' onclick="bringToFront('box1')"
  style='position:absolute; left:50px; top:50px;
  width:100px; height:100px; background:red; z-index:1;'>Box 1</div>
```

```
<div id='box2' onclick="bringToFront('box2')"
    style='position:absolute; left:80px; top:80px;
        width:100px; height:100px; background:blue; z-index:1;'>Box 2</div>
```

>> **Exam Tip:** z-index values are strings in JavaScript (e.g., '10').

8. Locating the Mouse Cursor

Property	Description
clientX, clientY	Position relative to the browser window (upper-left corner)
screenX, screenY	Position relative to the entire screen (upper-left corner)

Example Program: where.html

```
<script>
function showCoords(e) {
    document.getElementById('output').innerHTML =
        'Window X: ' + e.clientX + ', Y: ' + e.clientY + '<br/>' +
        'Screen X: ' + e.screenX + ', Y: ' + e.screenY;
}
</script>
<body onmousemove='showCoords(event) '>
    <div id='output'></div>
</body>
```

9. Reacting to a Mouse Click

The mousedown and mouseup events are used when more precise click control is needed.

Example Program: anywhere.html

```
<script>
function placeImage(e) {
    var img = document.getElementById('myImg');
    img.style.left = e.clientX + 'px';
    img.style.top = e.clientY + 'px';
    img.style.visibility = 'visible';
}
</script>
<body onmousedown='placeImage(event) '>
    <img id='myImg' src='star.png'
        style='position:absolute; visibility:hidden;' />
    <p>Click anywhere to place the image!</p>
</body>
```

10. Slow Movement (setTimeout and setInterval)

Function	Behaviour
setTimeout(code, ms)	Executes code ONCE after the given delay in milliseconds
setInterval(fn, ms)	Executes function REPEATEDLY at fixed intervals
clearInterval(timer)	Stops a running setInterval timer

Example Program: moveText.html

```
<script>
var currentX = 0;
var targetX = 300;
function moveText() {
    var txt = document.getElementById('movingText');
    if (currentX < targetX) {
        currentX += 5;
        txt.style.left = currentX + 'px';
        setTimeout('moveText()', 30); // calls itself after 30ms
    }
}
</script>
<body onload="setTimeout('moveText()', 100)">
    <p id='movingText'
        style='position:absolute; left:0px; top:100px; font-size:20px;'>
        I am moving slowly!
    </p>
</body>
```

>> Exam Tip: CSS position strings like '150px' must be parsed before arithmetic: `parseInt('150px')` gives 150.

11. Dragging and Dropping Elements

Drag-and-drop is implemented by combining three mouse events:

- mousedown (grabber): Records click offset; assigns mousemove and mouseup handlers.
- mousemove: Updates top and left based on new cursor position.
- mouseup (dropper): Removes the mousemove and mouseup handlers.

Example Program: dragNDrop.html

```
<script>
var offsetX, offsetY;
function grabber(e) {
    var elem = document.getElementById('dragBox');
    offsetX = e.clientX - parseInt(elem.style.left);
    offsetY = e.clientY - parseInt(elem.style.top);
    document.onmousemove = mover;
    document.onmouseup = dropper;
}
function mover(e) {
    var elem = document.getElementById('dragBox');
    elem.style.left = (e.clientX - offsetX) + 'px';
    elem.style.top = (e.clientY - offsetY) + 'px';
}
function dropper() {
    document.onmousemove = null;
    document.onmouseup = null;
}
</script>
<div id='dragBox'
    style='position:absolute; left:100px; top:100px;
        width:120px; height:60px; background:lightblue; cursor:move;'
    onmousedown='grabber(event)'>
    Drag Me!
</div>
```

UNIT 3 -- PART A

Introduction to PHP

1. Origin and Uses of PHP

- Developed by Rasmus Lerdorf in 1994
- PHP = Hypertext Preprocessor (recursive acronym)
- Server-side scripting language embedded in XHTML pages
- File extensions: .php, .phtml, .php3
- Good support for form processing, databases, distributed object architectures

2. Overview of PHP

When a .php document is requested: Server detects the extension, passes to PHP processor, which generates HTML output sent to browser. The client NEVER sees PHP code.

Two Modes of Operation

- Copy mode -- Plain HTML copied directly to output
- Interpret mode -- PHP code is executed; output sent to client

Characteristics

- Dynamic typing (untyped variables)
- Associative arrays
- Pattern matching (POSIX and Perl regex)
- Extensive standard libraries

3. General Syntactic Characteristics

```
<?php
// PHP code goes here
$name = 'John';
echo "Hello, $name!";
?>
```

- PHP code is enclosed between <?php and ?>
- Include files: include('filename.inc');
- All variable names begin with \$ (case-sensitive)
- Keywords and function names are NOT case-sensitive
- Statements end with semicolons
- Comments: // or # (single-line), /* */ (multi-line)

4. Primitives, Operations and Expressions

Category	Types
Scalar	boolean, integer, double, string
Compound	array, object

Category	Types
Special	resource, NULL

Variables

```
$x      = 10;           // integer
$price  = 3.14;         // double
$name   = 'Alice';      // string
$flag   = TRUE;         // boolean
isset($var)           // returns true if assigned
unset($var)            // sets back to NULL
```

String Type

```
$sum = 12;
echo "The sum is $sum";    // prints: The sum is 12  (variable interpolated)
echo 'The sum is $sum';    // prints: The sum is $sum (no interpolation)
```

Boolean Falsy Values

```
// All evaluate to FALSE:
0, "", "0", NULL, 0.0
```

Arithmetic and String Operations

```
12 / 6;    // 2    (integer -- no remainder)
12 / 5;    // 2.4  (double -- remainder exists)
$s = 'Hello' . ' World'; // String concat with .
strlen($s); // 11
$s{1};      // 'e'  (character access)
```

5. Output

```
print("Hello World");
printf("x = %5d is %s\n", $x, $size);
// %5d -> integer, min 5 chars wide
// %s   -> string
// %.2f -> double with 2 decimal places

// Example: today.php
<?php
    $today = date('l, F j, Y');
    print("<p>Today is $today</p>");
?>
```

6. Control Statements

if - elseif - else

```
<?php
    $score = 75;
    if ($score >= 90) {
        echo 'Grade: A';
    } elseif ($score >= 75) {
        echo 'Grade: B';
    } elseif ($score >= 60) {
        echo 'Grade: C';
    } else {
        echo 'Grade: F';
    }
}
```



```
?>
```

switch

```
$day = 'Mon';
switch($day) {
    case 'Mon': echo 'Monday'; break;
    case 'Tue': echo 'Tuesday'; break;
    default:    echo 'Other day';
}
```

Loop Statements

```
// while
$i = 1;
while ($i <= 5) { echo $i . '<br/>'; $i++; }

// for
for ($i = 1; $i <= 5; $i++) { echo $i . '<br/>'; }

// do-while
do { echo $i . '<br/>'; $i++; } while ($i <= 5);

// Sum of 1 to 100
$sum = 0;
for ($i = 1; $i <= 100; $i++) { $sum += $i; }
echo "Sum = $sum"; // Sum = 5050
```

7. Arrays

PHP arrays combine regular indexed arrays and associative (hash) arrays.

Array Creation

```
// Numerically indexed
$fruits = array('apple', 'banana', 'cherry');

// Associative (string indexed)
$capitals = array('India' => 'Delhi', 'France' => 'Paris');

// Direct assignment
$colors[] = 'red'; // auto-index 0
$colors[] = 'blue'; // auto-index 1
$colors['x'] = 'xerxes'; // string key
```

Accessing Elements and foreach

```
echo $fruits[0]; // apple
echo $capitals['India']; // Delhi

$low = array('Mon' => 23, 'Tue' => 18, 'Wed' => 27);
foreach ($low as $day => $temp) {
    echo "The low on $day was $temp <br/>";
}
```

Useful Array Functions

Function	Description
array_keys(\$arr)	Returns list of keys
array_values(\$arr)	Returns list of values

Function	Description
array_key_exists(\$k, \$arr)	Checks if key exists
implode(',', \$arr)	Joins array into string
explode(',', \$str)	Splits string into array
array_push(\$arr, \$val)	Appends value to array
array_pop(\$arr)	Removes and returns last element
sort(\$arr)	Sorts by value, reindexes
asort(\$arr)	Sorts by value, keeps keys
ksort(\$arr)	Sorts by key

8. Functions

```
function functionName($param1, $param2) {  
    // body  
    return $result;  
}
```

- Function names are NOT case-sensitive
- Parameters are passed by value by default
- Pass by reference: use & before parameter

Variable Scope

```
$globalVar = 100;  
function myFunc() {  
    global $globalVar;    // declare use of global variable  
    echo $globalVar;  
}
```

Static Variables

```
function counter() {  
    static $count = 0;    // retained across calls  
    $count++;  
    echo $count;  
}  
counter(); // 1  
counter(); // 2  
counter(); // 3
```

Example: Factorial Function

```
function factorial($n) {  
    if ($n <= 1) return 1;  
    return $n * factorial($n - 1);  
}  
echo factorial(5); // 120
```

9. Pattern Matching

Function	Description
preg_match(\$pattern, \$str)	Returns 1 if match found, 0 if not

Function	Description
<code>preg_split(\$pattern, \$str)</code>	Splits string by pattern, returns array
<code>preg_replace(\$p, \$repl, \$str)</code>	Replaces all matches

```
// Validate email
$email = 'test@example.com';
if (preg_match('/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/', $email)) {
    echo 'Valid email';
}

// Split on spaces or commas
$words = preg_split('/[\s,]+/', 'apple, banana orange');
foreach ($words as $w) echo $w . '<br/>';
```

10. Form Handling

PHP accesses form data through superglobal arrays: `$_POST` for method=post and `$_GET` for method=get.

HTML Form (order.html)

```
<form action='order.php' method='post'>
    Name:      <input type='text' name='custName' /><br/>
    Quantity: <input type='text' name='qty' /><br/>
    <input type='submit' value='Order' />
</form>
```

PHP Processor (order.php)

```
<?php
    $name  = $_POST['custName'];
    $qty   = $_POST['qty'];
    $price = 5.99;
    $total = $qty * $price;
    printf('<p>Order for %s: %d items = $%.2f</p>', $name, $qty, $total);
?>
```

11. Cookies

HTTP is stateless. Cookies help maintain state by storing key/value pairs on the browser, sent back with every request to the same server.

Setting a Cookie

```
<?php
// Must be called BEFORE any HTML output
setcookie('username', 'Alice', time() + 86400); // expires in 1 day
setcookie('lastVisit', date('Y-m-d'));
?>
```

Reading a Cookie

```
<?php
if (isset($_COOKIE['username'])) {
    echo 'Welcome back, ' . $_COOKIE['username'];
} else {
    echo 'Hello, new visitor!';
}
?>
```

Deleting a Cookie

```
setcookie('username', '', time() - 3600); // set expiry in the past
```

- Cookies are only returned to the server that created them
 - Browsers allow users to disable/delete cookies
 - Systems depending on cookies fail if cookies are disabled
-

12. Session Tracking

Sessions allow maintaining state across multiple pages. PHP generates a session ID to identify each session.

```
<?php
session_start(); // Must be FIRST line, before any output

// Store data
$_SESSION['user'] = 'Alice';
$_SESSION['cart'] = array('item1', 'item2');

// Read data
echo 'Hello ' . $_SESSION['user'];

// Destroy session
session_destroy();
?>
```

Feature	Cookie	Session
Storage	Client (browser)	Server
Security	Less secure	More secure
Lifetime	Set by programmer	Until browser closes (default)
Size limit	~4KB	Larger (server disk)

UNIT 3 -- PART B

XML

1. Introduction to XML

- XML = eXtensible Markup Language
- A simplified version of SGML (Standard Generalized Markup Language)
- A meta-markup language -- used to define other markup languages
- XML is NOT a replacement for HTML; XHTML is an XML-based version of HTML
- XML is a universal data interchange language
- In XML, tags carry the MEANING of their content
- Markup languages defined with XML are called XML applications

2. XML Syntax

Basic Rules

- XML declaration at top: `<?xml version='1.0' encoding='utf-8'?>`
- Every document has a single root element
- Every element with content must have a closing tag
- Empty elements: `<elementName />`
- XML is case-sensitive: `<Name>` differs from `<name>`
- Attribute values must be in quotes
- Comments: `<!-- comment here -->`

Example XML Document

```
<?xml version='1.0' encoding='utf-8'?>
<ad>
  <year>1960</year>
  <make>Cessna</make>
  <model>172</model>
  <location>
    <city>Chicago</city>
    <state>IL</state>
  </location>
</ad>
```

CDATA Sections

```
<description>
  <![CDATA[
    Content NOT parsed: <b>bold</b> & 'quotes' work here.
  ]]>
</description>
```

3. Document Type Definitions (DTD)

Keyword	Purpose
ELEMENT	Defines element structure
ATTLIST	Defines attributes of elements

Keyword	Purpose
ENTITY	Defines reusable content
NOTATION	Defines data type notations

Declaring Elements

```
<!ELEMENT memo (from, to, date, re)>

// Modifiers:
// + = one or more    * = zero or more    ? = optional
<!ELEMENT person (parent+, age, spouse?, sibling*)>

// Leaf node (content) types:
<!ELEMENT city (#PCDATA)>    // parsable character data
<!ELEMENT image EMPTY>      // no content
<!ELEMENT anything ANY>     // any content allowed
```

Declaring Attributes

```
<!ATTLIST person
    id      CDATA      #REQUIRED
    age     CDATA      #IMPLIED
    type    CDATA      'human'>
// #REQUIRED = must be given
// #IMPLIED  = optional, no default
// #FIXED v  = always fixed value
```

Declaring Entities

```
<!ENTITY st 'Sachin Tendulkar'>
// Used in XML as: &st; expands to: Sachin Tendulkar
```

Internal vs External DTD

```
<!-- Internal DTD -->
<!DOCTYPE memo [
    <!ELEMENT memo (from, to, date, body)>
    <!ELEMENT from (#PCDATA)>
    <!ELEMENT to   (#PCDATA)>
]>

<!-- External DTD -->
<!DOCTYPE memo SYSTEM 'memo.dtd'>
```

Complete DTD + XML Example

```
<?xml version='1.0'?>
<!DOCTYPE book [
    <!ELEMENT book (title, author, year)>
    <!ELEMENT title  (#PCDATA)>
    <!ELEMENT author  (#PCDATA)>
    <!ELEMENT year    (#PCDATA)>
]>
<book>
    <title>Web Technologies</title>
    <author>Sebesta</author>
    <year>2008</year>
</book>
```

4. Namespaces

When combining XML tag sets from different sources, naming conflicts arise. Namespaces resolve this by prefixing element names with a unique URI.

Namespace Declaration Syntax

```
// Single namespace
<birds xmlns:bd='http://www.birds.org/names/species'>
  <bd:sparrow>House Sparrow</bd:sparrow>
</birds>

// Multiple namespaces
<states xmlns='http://www.states-info.org/states'
  xmlns:cap='http://www.states-info.org/state-capitals'>
  <state>
    <name>South Dakota</name>
    <population>754844</population>
    <capital>
      <cap:name>Pierre</cap:name>
      <cap:population>12429</cap:population>
    </capital>
  </state>
</states>
```

- Default namespace (no prefix) applies to unprefixed elements
- Attributes are NOT included in namespaces

5. XML Schemas

Disadvantages of DTD -- why schemas were created

- DTDs written in a different syntax from XML (not parsed by XML processors)
- DTDs do not support data type constraints (everything is text)
- Two syntaxes to learn -- one for documents, one for structure

Schema Advantages

- Written in XML itself
- Supports data types (integer, date, string, decimal, etc.)
- Supports complex types with attributes and nested elements

Example XML Schema

```
<?xml version='1.0'?>
<xsd:schema xmlns:xsd='http://www.w3.org/2001/XMLSchema'>

  <xsd:element name='book'>
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name='title' type='xsd:string' />
        <xsd:element name='author' type='xsd:string' />
        <xsd:element name='year' type='xsd:integer' />
        <xsd:element name='price' type='xsd:decimal' />
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>

</xsd:schema>
```

6. Displaying XML Documents with CSS

An `xml-stylesheet` processing instruction links an XML document to a CSS file.

```
<?xml version='1.0'?>
<?xml-stylesheet type='text/css' href='planes.css'?>
<catalog>
  <plane>
    <year>1977</year>
    <make>Cessna</make>
  </plane>
</catalog>
```

planes.css

```
catalog { display: block; font-family: Arial; }
plane   { display: block; border: 1px solid black; margin: 10px; padding: 10px; }
year    { display: block; font-weight: bold; color: blue; }
make    { display: block; color: green; }
```

7. XSLT Style Sheets

XSL (Extensible Stylesheet Language) family:

- XSLT -- transforms XML documents into other formats (XHTML, text, etc.)
- XPath -- selects parts of an XML document and computes values
- XSL-FO -- specifies formatting for printed pages

XSLT Processing

```
XML Document + XSLT Stylesheet --> XSLT Processor --> Output (XHTML/text)
```

```
// Link XSLT stylesheet to XML:
<?xml-stylesheet type='text/xsl' href='stylesheet.xsl'?>
```

Example: `xslplane.xml` (Source Data)

```
<?xml version='1.0'?>
<?xml-stylesheet type='text/xsl' href='xslplane.xsl'?>
<plane>
  <year>1977</year>
  <make>Cessna</make>
  <model>Skyhawk</model>
  <color>Light blue and white</color>
</plane>
```

Example: `xslplane.xsl` (Single Record Transformation)

```
<?xml version='1.0'?>
<xsl:stylesheet
  xmlns:xsl='http://www.w3.org/1999/XSL/Transform'
  xmlns='http://www.w3.org/1999/xhtml'
  version='1.0'>

  <xsl:template match='/'>
    <html><body>
      <h2>Aircraft Information</h2>
      <xsl:apply-templates select='plane' />
    </body></html>
  </xsl:template>

  <xsl:template match='plane'>
    <p>
      <b>Year:</b> <xsl:value-of select='year' /> <br/>
      <b>Make:</b> <xsl:value-of select='make' /> <br/>
      <b>Model:</b> <xsl:value-of select='model' /> <br/>
    </p>
  </xsl:template>
</xsl:stylesheet>
```



```
</p>
</xsl:template>

</xsl:stylesheet>
```

Multiple Records with for-each and sort

```
<xsl:template match='/'>
  <html><body>
    <table border='1'>
      <tr><th>Year</th><th>Make</th><th>Model</th></tr>
      <xsl:for-each select='catalog/plane'>
        <xsl:sort select='year' data-type='number' />
        <tr>
          <td><xsl:value-of select='year' /></td>
          <td><xsl:value-of select='make' /></td>
          <td><xsl:value-of select='model' /></td>
        </tr>
      </xsl:for-each>
    </table>
  </body></html>
</xsl:template>
```

Quick Reference Tables

Unit 2 -- JavaScript DOM Summary

Topic	Key Concept / Code
Positioning	position: absolute / relative / static
Moving elements	element.style.left = x + 'px'; element.style.top = y + 'px'
Visibility	element.style.visibility = 'visible' or 'hidden'
Background color	element.style.backgroundColor = 'yellow'
Font size	element.style.fontSize = '18px'
Dynamic content	element.value = '...' or element.innerHTML = '...'
Stacking	element.style.zIndex = '10' (higher = on top)
Mouse position	event.clientX, event.clientY
Slow movement	setTimeout('fn()', ms) or setInterval(fn, ms)
Drag and Drop	mousedown -> grabber -> mousemove -> dropper on mouseup

Unit 3 (PHP) Summary

Topic	Key Concept / Code
Variables	Start with \$, case-sensitive (\$Name != \$name)
String concat	. operator: \$s = 'Hello' . ' World'
Output	print('text') or printf('%d %s', \$n, \$str)
Associative array	array('key' => 'value'); access with \$arr['key']
foreach loop	foreach (\$arr as \$k => \$v) { ... }
Functions	function name(\$p) { return val; }
Pattern matching	preg_match('/regex/', \$str) or preg_split(...)
Form data	\$_POST['fieldname'] or \$_GET['fieldname']
Cookies	setcookie('name','val',time()+86400); read via \$_COOKIE[]
Sessions	session_start(); then use \$_SESSION['key']

Unit 3 (XML) Summary

Topic	Key Concept / Syntax
XML Declaration	<?xml version='1.0' encoding='utf-8'?>
DTD element	<!ELEMENT name (child+, opt?)>
DTD attribute	<!ATTLIST elem attr CDATA #REQUIRED>
DTD entity	<!ENTITY name 'value'> used as &name;
Namespace	xmlns:prefix='URI'
XML Schema	Written in XML; supports int, string, decimal, date types
CSS display	<?xml-stylesheet type='text/css' href='file.css'?>

Topic	Key Concept / Syntax
XSLT link	<code><?xml-stylesheet type='text/xsl' href='file.xsl'?></code>
XSLT template	<code><xsl:template match='element'></code>
Output value	<code><xsl:value-of select='child'/></code>
Loop	<code><xsl:for-each select='path'></code>
Sort	<code><xsl:sort select='field' data-type='number'/></code>